AnyKey: A Key-Value SSD for All Workload Types

Chanyoung Park
Hanyang University
Ansan, Republic of Korea
chanyoung@hanyang.ac.kr

Jungho Lee
Hanyang University
Ansan, Republic of Korea
ljghoo32@hanyang.ac.kr

Chun-Yi Liu
Micron Technology Inc.
Folsom, CA, USA
cliue@micron.com

Kyungtae Kang
Hanyang University
Ansan, Republic of Korea
ktkang@hanyang.ac.kr

Mahmut Taylan Kandemir
Pennsylvania State University
University Park, PA, USA
mtk2@psu.edu

Wonil Choi
Hanyang University
Ansan, Republic of Korea
wonilchoi@hanyang.ac.kr

Abstract

Key-value solid-state drives (KV-SSDs) are considered as a potential storage solution for large-scale key-value (KV) store applications. Unfortunately, the existing KV-SSD designs are tuned for a specific type of workload, namely, those in which the size of the values are much larger than the size of the keys. Interestingly, there also exists another type of workload, in practice, in which the sizes of keys are relatively large. We re-evaluate the current KV-SSD designs using such unexplored workloads and document their significantly-degraded performance. Observing that the performance problem stems from the increased size of the metadata, we subsequently propose a novel KV-SSD design, called `AnyKey`, which prevents the size of the metadata from increasing under varying sizes of keys. Our detailed evaluation using a wide range of real-life workloads indicates that `AnyKey` outperforms the state-of-the-art KV-SSD design under different types of workloads with varying sizes of keys and values.

CCS Concepts: • **Information systems** → **Storage management**; **Flash memory**; **Key-value stores**.

Keywords: Key-value solid-state drive, log-structured merge-tree, storage management software, tail latency

ACM Reference Format:
Chanyoung Park, Jungho Lee, Chun-Yi Liu, Kyungtae Kang, Mahmut Taylan Kandemir, and Wonil Choi. 2025. AnyKey: A Key-Value SSD for All Workload Types. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '25), March 30–April 3, 2025, Rotterdam, Netherlands.* ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/3669940.3707279

ASPLOS '25, March 30–April 3, 2025, Rotterdam, Netherlands

ACM ISBN 979-8-4007-0698-1/25/03
https://doi.org/10.1145/3669940.3707279

1 Introduction

The explosive spread of KV store-based applications/services has spurred the development of a new storage type, namely, KV-SSDs [30–32, 36, 43, 51, 53, 64]. Compared to conventional setups where KV stores are managed in the host system while employing traditional block-based SSDs, KV-SSDs integrate a KV store directly within the SSD device. The need for KV-SSDs originate from their distinct advantages. First, KV-SSDs offer low-latency access to large volumes of data by streamlining the software stack, allowing applications to directly issue KV requests to the storage. Second, they improve scalability by reducing the CPU and memory requirements in the host system; simply adding more KV-SSDs to the storage system enhances scalability. Finally, KV-SSDs improve the bandwidth and device lifetime of underlying SSDs by tailoring the KV store in an SSD-aware fashion, such as reducing write amplification.

Despite the substantial potential benefits of KV-SSDs mentioned above, there has been a lack of recent updates on commodity KV-SSD products and their industry use cases. This is primarily because the performance of KV-SSDs still lags behind host-based KV stores that leverage the rich resources of the host system. However, there is a growing expectation for maturity in KV-SSD technologies across various large-scale computing domains, including data centers, cloud computing, and high-performance computing, where the management of very large and expanding KV stores is commonplace [3, 4, 28, 38], and ongoing efforts remain active.[1] The key to the success of KV-SSDs lies in effectively utilizing the limited internal resources of SSDs to meet the performance requirements, and our work is one of the academic contributions to this effort.

KV-SSDs have entirely different internal architecture and interface from the traditional block-based SSDs, since they store/retrieve KV pairs directly into/from the device [53].

[1] Unofficial sources indicate that manufacturers like Samsung and SK Hynix are actively working on developing high-performance KV-SSDs. We also observe that industry engineers have emphasized the future importance of KV-SSDs in forums (e.g., SNIA SDC) and meetings (e.g., NVMe community), while academic researchers continue to explore various KV-SSD designs in the literature.

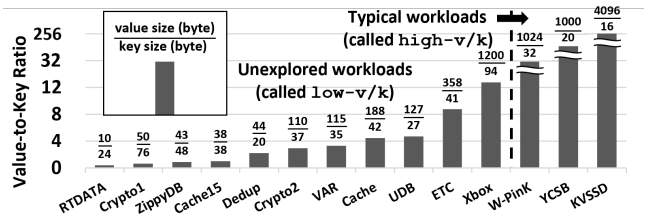


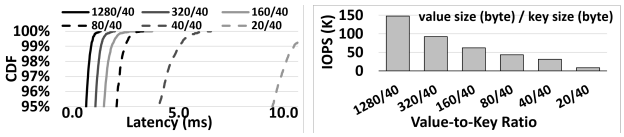
Figure 1. The newly-employed workloads with low value-to-key ratios (called `low-v/k`) along with three typical workloads with high value-to-key ratios (called `high-v/k`). The details of the workloads can be found in Table 2.

There exist various design choices for a KV-SSD, depending on the employed data structure for locating (indexing) KV pairs in the flash. While hash tables [32, 36, 64] and log-structured merge trees (LSM-trees) [30, 31, 43, 51] have been widely studied in literature, the latter seems to be the mainstream now, since its out-of-place update mechanism is flash-friendly and its multi-level indexing structure can guarantee bounded tail latencies. Note that, a state-of-the-art KV-SSD design, PinK [30, 31] also uses an LSM-tree.

While several LSM-tree-based KV-SSD designs have been proposed so far, unfortunately, they have not been validated using a large spectrum of potential KV workloads. Specifically, the existing designs use so-called "typical" workloads where the size of values is usually much larger than that of keys. For example, a state-of-the-art implementation [30] configures its workloads with 32-byte key and 1,024-byte value, and a widely-used benchmark suite, YCSB [17] specifies 20-byte key and 1,000-byte value by default. However, in reality, there exist another broad range of KV workloads where the ratio of value size to key size can be quite low. We studied such workloads with low value-to-key ratio from various domains[2], and show representative ones (along with the three typical workloads) in Figure 1. In this paper, we refer to the previously-studied workloads with high value-to-key ratios as `high-v/k`, whereas the previously-unexplored workloads with low value-to-key ratios are termed as `low-v/k`.

We find that the dominant LSM-tree-based KV-SSD designs experience poor performance under the `low-v/k` workloads. Specifically, we re-evaluated PinK [30] by varying the value-to-key ratios of a read-only workload. Figure 2 plots the tail latency and the IOPS achieved by PinK when the value size decreases from 1,280 bytes to 20 bytes while the key size (40 bytes) remains unchanged. The details of our experimental setup can be found in Section 5.1. It can be observed from these results that (a) the read latency (for the 95^{th} percentile) of PinK increases from 891μs to 8,904μs, and (b) the IOPS of PinK decreases by 94%, as the value-to-key ratio decreases from $\frac{1280}{40}$ to $\frac{20}{40}$. According to our analysis,

[2]These workloads are commonly encountered in various applications and services including databases [6], caches [59], social network services [10], and blockchains [9].



(a) 95^{th} percentile tail latency **(b)** IOPS

Figure 2. The observed performance of state-of-the-art PinK [30] under varying value-to-key ratios from $\frac{20}{40}$ to $\frac{1280}{40}$.

assuming that an SSD device is full of KV pairs, as the value-to-key ratio decreases (i.e., the relative size of keys increases), the size of the *metadata* (which indicates the locations of KV pairs in the flash) increases. This creates situations where the metadata are not fully accommodated in the device-internal DRAM, and consequently, each request may incur additional flash accesses before it actually reads the target KV pair. Note that, since other contemporary LSM-tree-based KV-SSDs [13, 43, 51] also share similar design principles with PinK, they would also suffer from the same problem.

Motivated by these observations, we propose a novel KV-SSD design, called **AnyKey**, which can minimize the size of metadata under our `low-v/k` workloads, and thus, locating target KV pair from the metadata can be done in the "device-internal DRAM", without incurring any extra flash accesses. Specifically, while the metadata in the conventional designs need to contain *all* the keys to indicate their KV pairs in the flash, AnyKey maintains the KV pairs in *sorted* groups, which allows the metadata to hold only one key for each group of KV pairs. While such a change can bring a significant overhead in the management of the LSM-tree (i.e., merging KV pairs in two neighboring levels) by relocating all the values for the KV pairs in each group, AnyKey can still achieve high IOPS by storing the values for the (recently/frequently-updated) keys in a separate flash area, and thus, preventing them from being moved during the LSM-tree management process.

Observing that existing designs were validated in different environments and settings, we constructed an emulation-based KV-SSD development and evaluation environment, based on a publicly-available flash emulator (FEMU [45]) and a widely-employed host system driver (uNVMe [2]), in which we implemented and evaluated PinK, AnyKey, and its enhanced version. Our detailed experimental evaluation demonstrates that, compared to PinK, AnyKey improves the 95^{th} percentile tail latencies and the IOPS by 56% and 299%, respectively, for our eleven `low-v/k` workloads. Also, the enhanced version of AnyKey achieves 315% and 15% higher IOPS than PinK for the eleven `low-v/k` and three `high-v/k` workloads, respectively. Overall, this paper makes the following main **contributions**:

- We unveil a range of real KV workloads with *low* ratios of the value size to the key size from various domains, and find that the existing KV-SSD designs experience significantly-degraded performance under such `low-v/k` workloads.

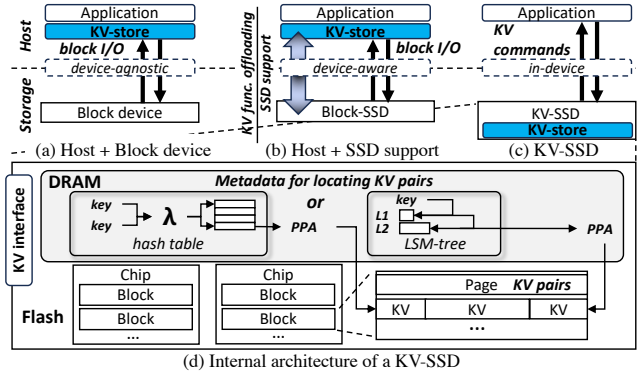


Figure 3. (a-c) The comparison of three execution models for a KV store, and (d) the internal architecture of a KV-SSD.

• Focusing on the `low-v/k` workloads, we propose and evaluate a novel KV-SSD design, called `AnyKey`, which can improve the tail latencies, IOPS, and device lifetime significantly, compared to the state-of-the-art. The key idea behind `AnyKey` is to streamline the metadata for locating KV pairs and make it fit into the device-internal DRAM.
• We also present an enhanced version of `AnyKey`, which can improve IOPS significantly for the `high-v/k` workloads, over the state-of-the-art, by addressing the high overheads that can come from the management of the LSM-tree under those workloads. Consequently, the advanced `AnyKey` outperforms the state-of-the-art for *all* types of workloads.

2 Preliminaries and Related Work

2.1 Execution Models of KV Store

There are three different models for hosting a KV store in a target system, depending on how the storage is used.

Host + Block device: As shown in Figure 3(a), one can operate a KV store in the host system with one or multiple conventional block devices. RocksDB [22] and LevelDB [27] are representative KV stores that come under this category. While this traditional execution model does not require any change in the system, it usually brings a performance issue, as it needs to generate block I/O to access the underlying storage device through the typical S/W stack. A body of prior works [7, 11, 18–20, 44, 46–49, 61] have tried to improve performance by best leveraging the underlying block device. In such a context, another body of recent works [12, 16, 23, 35, 37, 54, 56, 60] have introduced byte-addressable persistent memory technology in the host to reduce the amount of block I/O traffic.

Host + SSD support: To address the limited performance issue originating from employing the cumbersome, full S/W stack and treating the underlying SSD as a black box, another body of works [42, 50, 57, 62, 63] have attempted to take advantage of the unique features or structures/behavior of the SSD, as illustrated in Figure 3(b). For instance, one study defined and used new SSD interfaces that can leverage high levels of device-internal parallelism or transactional persistence capabilities of flash commands [50], whereas another work, which targets an Open-Channel SSD, modified the kernel driver to improve the caching and scheduling mechanisms [63].

KV-SSD: A recent approach to operate a KV store is to bring it into the SSD itself, and let the SSD to manage the entire KV store, as described in Figure 3(c). Doing so can offer higher performance, better scalability, and more efficient device management than the other two methods described above [4, 36, 38]. There have been various KV-SSD designs with different internal structures, behaviors, and interfaces [8, 13, 24, 30–32, 36, 43, 51, 64], each trying to overcome the limited resources within the SSD and/or take advantage of the SSD-specific characteristics. Note that this paper introduces and evaluates a novel KV-SSD design.

2.2 Existing KV-SSD Designs

Hardware architecture: A KV-SSD has the same hardware architecture that is commonly-employed in the conventional block-based SSD. As depicted in Figure 3(d), its architecture involves a few flash chips, each of which includes thousands of blocks (erase unit), and each block consists of hundreds of pages (read/write unit). Since KV pairs are directly stored in the pages, a KV-SSD also needs to store *metadata* to locate the target KV pair, and such metadata are usually kept in the device-internal DRAM. Among various possible data structures to manage the metadata, *hash tables* or *LSM-trees* are the most popular ones.

Hash-based KV-SSD: The KV-SSD design in its early stage was mostly based on a hash table for the metadata due to its simplicity [8, 24, 32, 36, 64]. However, using the hash table within an SSD has two serious issues. First, its in-place update mechanism generates partial page updates under writes, which invokes frequent garbage collection (GC) operations, and in turn, wastes bandwidth and device lifetime. Second, while minimizing the hash table to make it fit into the device-internal DRAM can avoid the partial update problem, such a small hash table causes frequent hash collision for reads, and consequently, increases tail latencies. Note that other sorted indexing structures, such as B-trees [15] and learned indexes [18, 40], may also be inappropriate, since they also require in-place updates, and consequently, lead to device management issues for SSDs under write-heavy workloads.

LSM-tree-based KV-SSD: As an alternative, LSM-tree has been gaining popularity, due to its (flash-friendly) out-of-place update mechanism. Note that the recently-presented KV-SSD designs are all based on an LSM-tree [13, 30, 31, 43, 51], and our proposed design also employs an LSM-tree.

3 A Re-evaluation of the Current Design

We re-evaluate the current design of LSM-tree-based KV-SSDs using the `low-v/k` workloads, from which we unveil its

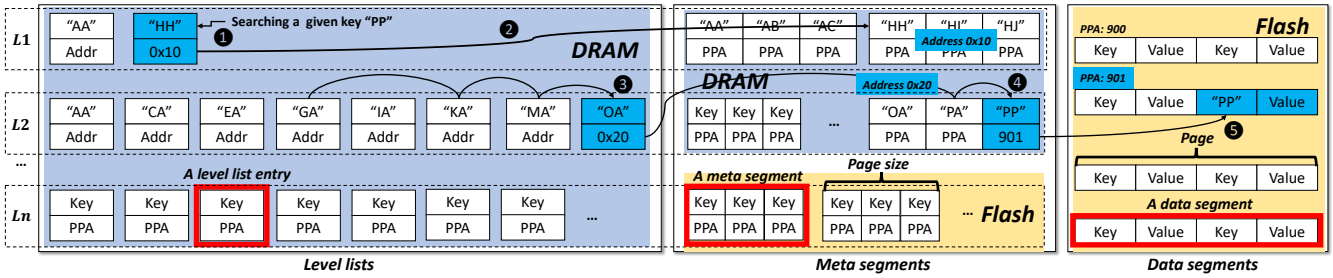
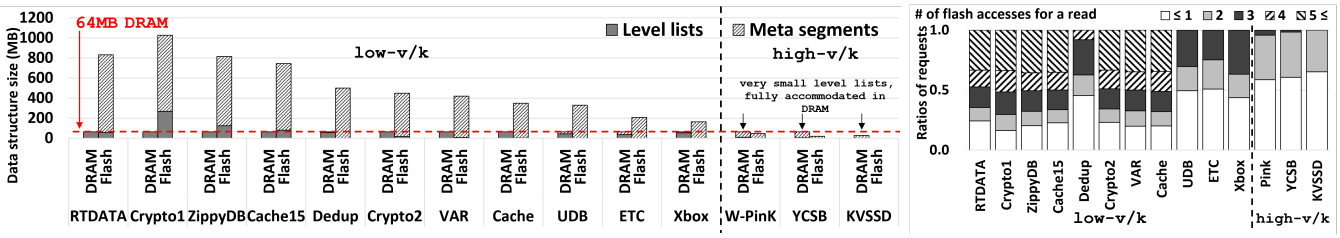


Figure 4. The internal data structures of PinK, where (i) the level lists and (ii) the meta segments hold the metadata for locating the KV pairs in (iii) the data segments.



(a) The sizes and locations (DRAM or flash) of the two metadata types. **(b)** The number of flash accesses for a request.

Figure 5. The contrasting behaviors of PinK under the three `high-v/k` and eleven `low-v/k` workloads.

structural limitations. While we pick state-of-the-art PinK [30] as a representative case, other contemporary LSM-tree-based KV-SSDs also have similar design principles.

3.1 Structure and Operations of PinK

The internal structures of KV-SSDs differ from those of general LSM-tree-based KV stores. Specifically, general LSM-trees sort and maintain KV pairs in *files* known as sorted string tables (SSTables). In contrast, KV-SSDs, which have files physically scattered across flash, require new types of data structures that can collectively function like SSTables.

Figure 4 illustrates the internal structure of PinK, and how a target KV pair is located. While actual KV pairs are stored in flash as *data segments*, their keys and pointers to the corresponding KV pairs are sorted within *meta segments* and *level lists*. These meta segments and level lists, which maintain the metadata, are kept in the device-internal DRAM.

Data segment: All KV pairs, which are stored in the flash, are managed in the data segments. Simply put, each flash page can be a data segment where multiple KV pairs are co-located.

Meta segment: A meta segment, which can fit into a flash page, maintains *sorted* keys and the physical flash addresses (i.e., physical page addresses (PPAs)) of the corresponding KV pairs. Due to the limited DRAM capacity, PinK keeps the meta segments in the upper levels of the LSM-tree in the device-internal DRAM, while storing those in the lower levels in the flash (pages).

Level list: For each meta segment, there exists a level list entry that includes the first (or smallest) key of the meta segment and the location of the meta segment (i.e., the DRAM address or the flash address, depending on its current location). These level list entries are grouped into multiple levels (L_1 to L_n), where the entries within each level list are sorted.

Procedure of locating a KV pair (read operation): Figure 4 shows an example of handling a key request "PP". Given the key, it is searched from the level lists starting from L_1. If it is not found in the current level list, the next level list is checked. ❶ Once the level list entry into which "PP" falls is found, its corresponding meta segment is read in the DRAM address of 0x10. ❷ Note however that "PP" may not exist in the meta segment, since the key ranges of the level lists can overlap across different levels; in such a case, the next level is searched. ❸ If a level list entry is found in L_2 and ❹ the corresponding meta segment in the DRAM address of 0x20 actually includes "PP", the location of the data segment where the target KV pair is stored is obtained. ❺ Finally, the data segment is read in the flash address of 901, and the target value is extracted.

Procedure of updating on an existing key or adding a new key (write operation): Initially, a write request is buffered in the device-internal DRAM. When the buffer becomes full, its KV pairs are merged into L_1. This process of creating a new L_1 involves generating new level list entries and meta segments in DRAM for the corresponding keys, while the values are written into data segments in flash. In simpler terms, each write request is serviced immediately from the buffer, and later, it becomes part of the LSM-tree.

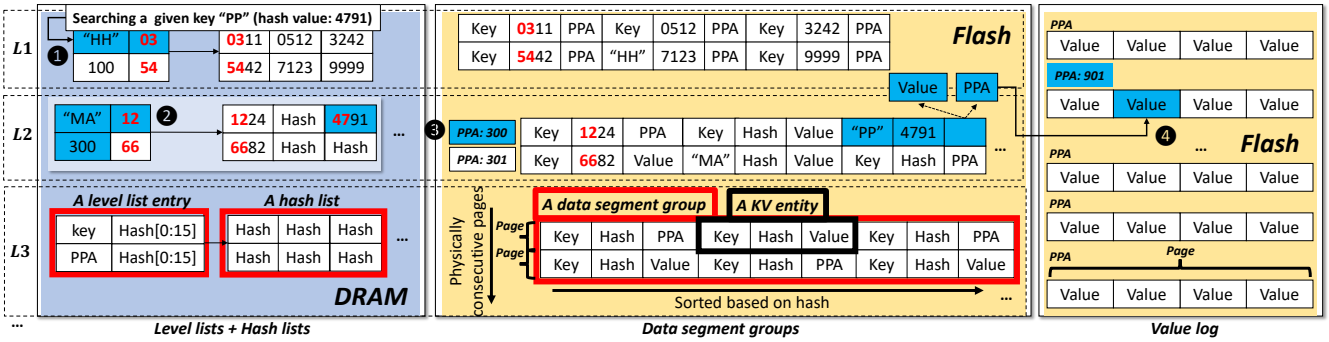


Figure 6. The four major data structures in AnyKey: the two types of metadata, (i) the level lists and (ii) the hash lists, are kept in the device-internal DRAM, whereas (iii) the data segment groups and (iv) the value log in the flash hold target values.

3.2 Analysis on the Metadata Size of PinK

We examine the behavior of PinK under the employed workloads, assuming that a 64GB SSD with a 64MB DRAM[3] is full of the KV pairs. Figure 5a shows the sizes and the locations (i.e., in the device-internal DRAM or in the flash) of (i) the level lists and (ii) the meta segments, and Figure 5b gives the distribution of requests based on their flash access counts. We can make the following two contrasting observations:

- **Under `high-v/k` workloads:** As demonstrated in [30, 31], under the three `high-v/k` workloads, due to their small key sizes, the collective size of the level lists and the meta segments (which contain keys) is quite small, and most of them are accommodated in the DRAM. As a result, locating a key can be done mostly within the DRAM, and the number of flash accesses for a request can be limited to at most 3.
- **Under `low-v/k` workloads:** As the value-to-key ratio decreases (i.e., the relative size of keys increases), the cumulative size of the level lists and the meta segments increases significantly. As a result, they cannot be fully accommodated in the fixed size of the DRAM, and parts of them need to be stored in the flash. For the workloads with relatively high value-to-key ratios (e.g., `UDB`, `ETC`, and `Xbox`), only the meta segments are stored in the flash, which may require at most one or two more flash accesses for reading them to locate KV pairs. In contrast, for the workloads with relatively low value-to-key ratios (e.g., `Crypto1`, `ZippyDB`, and `Cache15`), in addition to the entire the meta segment, some level lists are also stored in the flash. In such situations, searching the requested key involves several additional flash reads for the level lists in the flash, which can significantly hurt the overall performance.

4 The Proposed KV-SSD Design

4.1 Design Principles and Overall Structure

4.1.1 Minimizing the Size of the Metadata. To reduce the size of the metadata, we propose to manage a set of KV pairs in a *group*, in which the KV pairs are sorted. Existing KV-SSD designs, in practice, require each of the KV pairs to have its own metadata to directly locate it in the flash. Against such a design choice, if a number of KV pairs are sorted and managed as a group, a target KV pair within the group can be located by using only the metadata for the group. Keeping this principle in mind, we design "new data structures" for KV pairs and their metadata. Figure 6 illustrates the internal structure of our proposed `AnyKey`.

Note that our approach differs from the traditional method of keeping KV pairs sorted in general LSM-trees. In LSM-trees, when data in DRAM are flushed to disk, they are sorted and stored in files called sorted string tables (SSTables) [27, 52]. Here, LSM-trees were originally designed for HDDs, where KV pairs in SSTables could be located contiguously. However, on SSDs, logically contiguous data may be physically scattered depending on the page allocation strategies used [33]. In contrast, our approach stores a large number of KV pairs directly in flash, keeping them sorted.

Data segment group (in the flash): We combine multiple flash pages to form a "data segment group" where multiple KV pairs are sorted. Doing so can reduce the collective size of the metadata significantly, as it requires metadata only for each data segment group. Further, such a design still enables us to locate a target KV pair using only the group metadata, since the KV pairs in each group are sorted. While there can exist different ways of grouping flash pages, in our current implementation, we integrate the neighboring physical pages within a flash block. Since the PPAs of the physical pages within a block are sequential, doing so can help us access to any flash page within a data segment group with only the PPA of the first page in the group.

Once the target data segment group is identified, it is critical to efficiently locate the flash page where the target KV pair is stored. Otherwise, all the flash pages in the data segment group may have to be read. While one simple approach for identifying a KV pair within a data segment group would be keeping the first (smallest) key of each flash page in the group metadata, such an approach could increase the size

[3]Note that the DRAM capacity is typically set to 0.1% of the total SSD capacity [25, 39], though this can actually vary in real products.

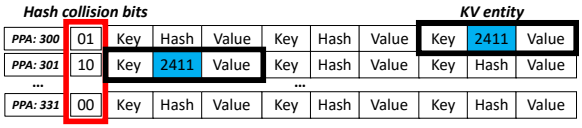


Figure 7. When two different keys have the same hash value.

of the metadata significantly when the size of the keys is relatively large (i.e., under the `low-v/k` workloads). To this end, we introduce small-sized *hash values* for the large-sized keys and use them to sort the KV pairs within a data segment group. Note that the use of hash values in our design is intended to reduce metadata size. The number of hash values that must be maintained in DRAM is quite small, compared to hash-based KV-SSDs where each key has a corresponding hash value as metadata.

Figure 6 describes a data segment group, where 2 flash pages are included and 6 KV pairs are stored. Each KV pair in the data segment group is stored in the form of {(i) the key, (ii) the hash value of the key, (iii) the value}, which we call the *KV entity*. For (ii) the hash values, we observe from our analysis that, using 32-bit hash values can uniquely identify a key *in a data segment group* where up to several thousands of KV pairs are stored. For (iii) the value, as can be seen in some KV entities, a PPA can be stored instead, which will be explained later.

If different keys within a data segment group have the same hash value by chance, since such KV entities are stored *consecutively*, there can be two possible cases depending on their locations: (i) in the same page or (ii) in two consecutive pages. For the former, the target KV entity can be easily identified by reading the page and checking their keys. For the latter, both the pages should be read, since we have no idea which page includes the target KV entity. Figure 7 illustrates an example scenario where two KV entities have the same hash value "2411" in two consecutive pages (PPAs 300 and 301). Once a KV entity is found at the beginning or at the end of a page, the previous page or the next page may hold a KV entity with the same hash value. To this end, for each page, we reserve 2 bits, called *hash collision bits*, and set them to "01" if the next page needs to be read, and to "10" if the previous page should be read. Note however that, according to our analysis, such a hash collision situation occurs only in 0.075% of the entire groups constructed during the execution, and thus, its impact on the overall performance is negligible. Nevertheless, our experimental evaluations include such scenarios as well.

In the current implementation, we combine 32 8KB flash pages into a data segment group, where 100–5,000 KV pairs can be accommodated under our tested workloads. As more flash pages are combined in a data segment group (resulting in fewer groups in an SSD), the total size of the group metadata decreases. However, this reduction in size reaches a limit, since the group metadata (which will be discussed later) includes a fixed-sized hash value for each page in the group, indicating the first key in the page, and the total number of pages in an SSD remains constant. Furthermore, a very large data segment group may experience hash collisions, which leads to the need to read multiple pages to locate the target KV pair within the group. Note that the number of pages in a data segment group can be determined by considering its impact on the metadata size and the indicated performance in device configurations (e.g., the number of pages, the size of the DRAM, etc).

Level lists (in the DRAM): Given that we manage multiple KV pairs within a data segment group, a careful design of the corresponding group metadata is required. While we can use level lists to accommodate such group metadata as in other LSM-tree-based designs, each level list entry should be newly-designed to include the metadata of the corresponding data segment group.

Figure 6 depicts a level list entry (in the DRAM) that corresponds to a data segment group, which is in the form of {(i) the smallest key, (ii) the PPA of the first page, (iii) a list of the hash values of the first keys in the pages}. For a target KV pair, its data segment group can be identified using only (i) the smallest key, since the level list entries are sorted at each level. Once the target data segment group is identified, all the flash pages within it can be accessed using (ii) the PPA of the first page. To find the exact flash page where the target KV pair is stored, we can take advantage of (iii) the list of hash values for the first keys in the pages. Since all the KV pairs are sorted based on the hash values of the keys in a data segment group, the target page can be easily found by searching the hash value of the target key from the list of hash values. In the current implementation, we further reduce the size of the level list entry by using only the first 16 bits of the hash values (Hash[0:15]).[4]

4.1.2 Further Reducing Flash Accesses.

The combination of the data segment groups and the level lists may require additional flash accesses when a target KV pair is located. This is because (i) the level lists provide only the ranges of keys, and (ii) the ranges of keys can overlap across different levels, and thus, even if the key falls into a level list entry, the corresponding data segment group may *not* hold the target KV pair. We want to emphasize that this problem also exist in other LSM-tree-based designs. To address this problem in our design, we add a mechanism to check whether the target KV pair is indeed in a data segment group.

Hash lists (in the DRAM): We introduce a new metadata type in the LSM-tree, which is called "hash list". Each level list entry has a hash list which maintains the hash values of

[4]Note that those Hash[0:15] values are used only for identifying one from 32 hash values, each corresponding to the first KV pair in each page. In our analysis, only 0.01% of the total level list entries constructed during the execution include the same Hash[0:15] values. For such entries, we use the original 32-bit hash values.

all the keys that actually exist in the corresponding data segment group. Figure 6 describes a hash list that corresponds to a level list entry in the DRAM. If the hash value of the target key is not found in the hash list, this indicates that the target KV pair does not exist in the data segment group, thus searching for the key continues in the next level lists.

We use the remaining DRAM space to keep the hash lists, which enables to check whether a KV pair is in a data segment group before reading the target flash page. If there is no more available space in the DRAM, we do not maintain the hash lists for the lower level lists. For those level lists, when the problematic situations occur, AnyKey cannot avoid additional flash accesses until the target KV pair is actually read. We want to re-iterate that this also occurs in other LSM-tree-based designs, and AnyKey reduces the chances of its occurrence by utilizing the remaining DRAM space. Since our approach of grouping a large number of KV pairs can significantly reduce the size of metadata (level lists), introducing a new metadata type (hash lists) in the available DRAM space can further enhance performance.

In our current implementation, we use an integer array for each hash list and a binary search for searching a hash value from it. One might consider using Bloom or Cuckoo filters to verify whether the requested key is present in the found level list, which could save DRAM space and reduce search time. However, our analysis indicates that generating these filters requires intensive computation, which places a significant burden on SSD-internal resources. Note that PinK [31] also identified this issue and chose not to include such filters in their design.

4.1.3 Improving the Efficiency of the LSM-tree Management.

While the proposed design described so far can improve the read latency, it can also increase the overhead of managing the LSM-tree. Specifically, as the level lists directly indicate the data segment groups where both keys and values are stored, during the compaction process[5], all the values in the data segment groups of the two target levels are read and written, wasting the device bandwidth and lifetime significantly.

Value log (in the flash): To prevent all the values in the data segment groups from being moved during the compaction process, we detach the values from the corresponding keys in the data segment groups and manage them separately. To this end, inspired by [49], we introduce a new data structure, called "value log", in the flash, where only values are written. As depicted in Figure 6, the flash space in AnyKey is divided into two areas – one for the data segment groups and the other for the value log. Therefore, the value of each KV pair can be stored in one of the two areas: in the KV entity of the data segment group or in the value log. In the latter case, the KV entity contains a pointer to where the value is stored in the value log (instead of the value itself). At first, all the values of writes are written into the value log, and then later, some of them are merged into the data segment groups to secure available space in the value log and continue to service the writes. The process of moving the values from the value log to the data segment groups is performed during the compaction process, which will be described in detail later in Section 4.2.

AnyKey reserves only the half of the remaining SSD capacity for the value log area. This is because an LSM-tree can increase as new KV pairs are added into the store. Even if all the values stored in the value log belong to such new KV pairs, AnyKey can still take advantage of the other half of the remaining capacity to merge them into the data segment groups. While we employ such a design choice, we also vary the size of value log to conduct a sensitivity study (Section 5.8). If there is no more available capacity in an SSD, AnyKey can employ half of the over-provisioned (OP) capacity[6] for the value log; according to our analysis, such a small value log is still beneficial. Note that this does not mean that we reserve the OP for accommodating more KV pairs or for storing our data structures. Since the OP is always used to accommodate updated values when the capacity is full, we keep those values in the OP, but in the form of the value log (instead of the data segment groups).

One might be concerned that values stored in the value log could remain in the same location without being relocated, which potentially leads to read errors over time. Note however that the purpose of introducing the value log in AnyKey is to address update-intensive workloads (such as those with a 20% write ratio, as used in our evaluation), where improving the efficiency of compaction operations is crucial. Consequently, values in the value log are likely to be updated before they experience read errors.

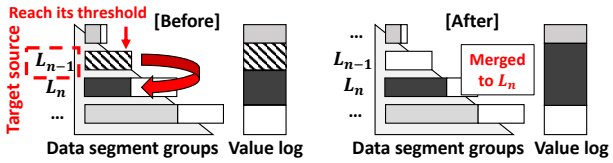
4.2 Detailed Operations

4.2.1 Read.

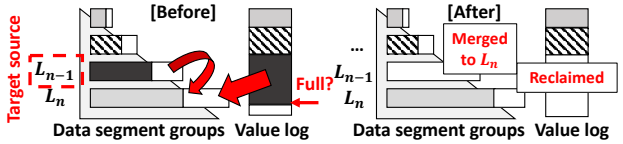
Given a key request, AnyKey uses the level lists and the hash lists to find the target data segment group, the KV entity within it, and the target value. If the KV entity holds the location information of the target value, AnyKey reads the value from the value log. Figure 6 describes an example scenario of reading the value for a key request. ❶ Given a key "PP" (its hash value is "4791"), AnyKey searches for the key in the level lists from L_1 to L_n, one after the other. If an entry where "PP" falls into the range of keys is detected, AnyKey checks whether its hash value ("4791")

[5] As key inserts and updates are serviced, due to its out-of-place update mechanism, an LSM-tree needs to maintain its structure by merging neighboring levels periodically, which is called *compaction* [52]. During a compaction process, two target levels, L_{n-1} (source) and L_n (destination), are merged (i.e., read, sorted based on keys, and written) into L_n. As a result, L_{n-1} becomes empty and a new sorted L_n is generated.

[6] The storage capacity that an SSD actually posses is slightly in excess of the vendor-advertised (and user-perceived) capacity; this excess amount is referred to as "over-provisioned" (OP) capacity.



(a) Tree-triggered compaction.



(b) Log-triggered compaction.

Figure 8. The two types of compaction operations.

is in the corresponding hash list. If the hash value is not found, this means that the target KV entity does not exist in the data segment group indicated by the level list entry. ❷ Then, AnyKey continues to search for "PP" in the next level. If another level list entry is selected as a candidate and its hash value is also found in the the corresponding hash list, the target data segment group is identified. ❸ Among the flash pages in the group, the first one in PPA 300 is selected and read, since the first 16 bits of the hash value ("47") fall between the first two (16-bit) hash values ("12" and "66") in the level list entry. If the target KV entity holds the value, we are done. ❹ Otherwise, the actual location of the value is extracted from the entity, and the page in the value log (PPA 901) is read.

4.2.2 Write. As in most KV-SSD designs [13, 30, 31, 43, 51], at first, a write request (an update on the existing key or a newly-added key) is buffered in the device-internal DRAM.[7] When the buffer becomes full, its KV pairs are merged into L_1 (i.e., L_0-to-L_1 compaction). Specifically, new data segment groups and level list entries for L_1 are constructed (all the values are written into the value log). Simply put, new values which join the LSM-tree are written into the value log.

4.2.3 Compaction. Similar to the compaction process employed by the traditional LSM-tree-based KV-SSDs, an L_{n-1}-to-L_n compaction in AnyKey also (i) reads all the KV pairs from both the levels, (ii) sorts them based on the keys, and (iii) constructs new data segment groups and the corresponding level list entries for L_n by cutting them off in order. However, the compaction process of AnyKey has two distinctive aspects. First, during (i) (i.e., when the KV pairs of the target levels are read), the value of each KV pair is read from where it is currently stored, which can be either a data segment group or the value log. Second, during (iii) (i.e., when each data segment group is created and written into the flash), the KV entities are sorted again based on the hash values of the keys. A compaction is invoked under the following two conditions: (i) when the size of a level exceeds its threshold (called *tree-triggered compaction*) and (ii) when the value log becomes full (called *log-triggered compaction*). Figure 8 illustrates the compaction operations adopted in AnyKey.

Tree-triggered compaction: When the size of L_{n-1} of the LSM-tree (i.e., the collective size of its data segment groups and the referenced values in the value log) reaches its threshold, a L_{n-1}-to-L_n compaction is invoked. Note that this is the conventional compaction operation, needed to maintain an LSM-tree with an out-of-place update mechanism.

Log-triggered compaction: AnyKey also invokes a compaction for the purpose of securing the available space in the value log. When the value log becomes full, a target (source) level L_{n-1} is selected, and a L_{n-1}-to-L_n compaction is invoked. During the compaction process, all the values of L_{n-1} and L_n in the value log are relocated into the data segment groups of the newly-created L_n, thus creating available space in the value log. For the target level L_{n-1}, AnyKey picks a level whose size of its values in the value log is the largest, since doing so would be the most beneficial in reclaiming the free space in the value log.

It is important to note that when both types of compaction operations are triggered and the level lists are modified, AnyKey checks the available DRAM space and updates the hash lists of the top levels accordingly. Since a compaction operation involves numerous flash reads and writes, updating the hash lists in DRAM incurs minimal overhead and can be effectively hidden. Our experimental results include the process of updating the hash lists.

4.2.4 GC. While a conventional GC operation relocates all the valid data (in the unit of a page) within a victim block and just erases the block, the GC operation in AnyKey (i) relocates each data segment group (in the unit of multiple pages) of the victim block, and after the relocation of a group, (ii) its new PPA is updated in the corresponding level list entry. According to our observations, most victim blocks in AnyKey do not include any valid data, and thus, the GC operation can erase them right away without the need of time-consuming process of relocating valid data, which renders the overall GC overhead quite low. This is because there is a tendency that the data segment groups of a level are stored in the same block, and those groups are relocated together during (both types of) compaction operations. Note also that the GC operation is not triggered in the value log since reclaiming flash blocks in the value log is performed during log-triggered compaction.

4.2.5 Range query. Since the KV pairs are sorted based on their hash values within each data segment group, given a range query, re-sorting them based on their keys requires a considerable amount of computations. To address this, we keep extra information to locate each KV pair in a data segment group (i.e., {target page, page offset} for each key),

[7] To our knowledge, the existing LSM-tree-based KV-SSD designs commonly employ a tiny write buffer, which is called L_0. AnyKey also inherits the general LSM-tree structure and maintains a small write buffer.

Table 1. The analysis on the metadata size in PinK and AnyKey under varying (low) value-to-key ratios, assuming that our 64GB SSD is full of KV pairs (Section 5.1).

Workload	PinK			AnyKey		
Value-to-key ratio (V/K)	Level lists	Meta segments	Sum	Level lists	Hash lists	Sum
4.0 (160B/40B)	56MB	316MB	372MB	29MB	35MB	64MB
2.0 (120B/60B)	117MB	414MB	531MB	34MB	30MB	64MB
1.0 (80B/80B)	200MB	503MB	703MB	38MB	26MB	64MB

which are sorted based on their *keys* and stored in the first page of the data segment group. Using such information, the KV pairs requested by a range query can be located quickly without any sorting process.

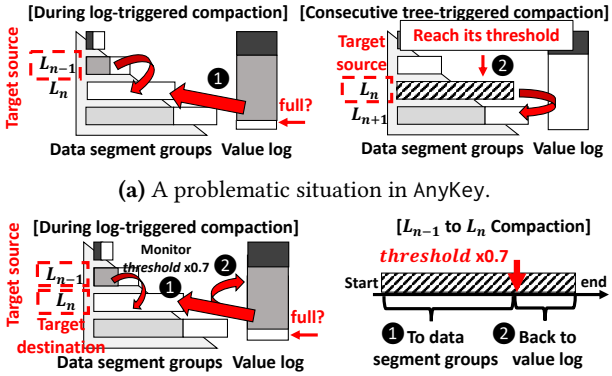
We want to emphasize that the performance of range queries in AnyKey improves as the size of range queries increases. This is because the performance of range queries depends on the number of flash pages that need to be read, and AnyKey requires fewer pages to be read for longer range queries compared to PinK. Specifically, the KV pairs requested by a range query are stored together in the flash pages of a data segment group, and consequently, reading them involves accessing only a few flash pages. In contrast, in PinK, although keys and pointers to their values are sorted in meta segments in DRAM, the corresponding KV pairs in data segments are scattered across different flash pages. As a result, range queries in PinK require reading a substantial number of flash pages. For small range queries, AnyKey also exhibits performance comparable to PinK, since it can directly locate target KV pairs using the location information stored in the first page without requiring to sort KV pairs based by keys. We report the performance of range queries in AnyKey in Section 5.7.

4.3 Analysis on the Size of Metadata

We perform a quantitative analysis on the size of the metadata to see how much AnyKey can reduce it, compared to PinK. Table 1 compares the sizes of the data structures for the metadata in PinK and AnyKey under varying value-to-key ratios. We make the following observations: (i) (when the value-to-key ratio is relatively low) the collective size of the metadata in AnyKey is much smaller than that of PinK. (ii) This is because the size of the level lists in AnyKey is reduced significantly, and its hash lists are generated using only the remaining DRAM space, limiting the collective size of the two data structures to the DRAM size. (iii) Consequently, even though the value-to-key ratio decreases extremely, AnyKey prevents the collective size of the metadata from increasing. The key knob to reduce the metadata size in AnyKey is to generate the metadata for each group of multiple KV pairs (i.e., data segment group).

4.4 Design Overhead

4.4.1 Storage Capacity Overhead. In AnyKey, each KV entity in a data segment group includes a 32-bit hash value of its key. If there exist ten million KV pairs in a 64GB SSD, the total size of the 32-bit hash values is 38MB, which is 0.05% of the entire SSD capacity. Also, each data segment group requires to maintain the location of all the KV pairs within the group for prompt range query processing. Assuming there exist 1,000 KV pairs in a group including 32 8KB-pages, the total size of such information is 2KB, which is less than 1% of the entire group size. We want to emphasize that such collective overhead is extremely small, compared to PinK, which may require to store two different copies for each key in the flash, if the meta segments could not be accommodated in the DRAM. We show, in Section 5.4, that the storage utilization (the contribution of the total size of unique KV pairs to the entire storage capacity) of AnyKey is higher than that of PinK.



(a) A problematic situation in AnyKey.

(b) The modified log-triggered compaction.

Figure 9. An illustration of the problematic situations during the compaction process, and an enhancement to avoid them.

4.4.2 Computation Overhead. Employing hash values in a KV-SSD design introduces its related computational tasks in its device. First, AnyKey needs to *generate the 32-bit hash value for a key*, when (i) a KV pair is written into the LSM-tree for the first time, and (ii) a read request is given. In the current implementation, we employed xxHash [14], which takes 79ns for generating a 32-bit hash value from a 40-byte key on a 1.2 GHz ARM Cortex-A53 core (which is a widely-employed compute resource in contemporary SSDs [5]). This is a marginal overhead, compared to the typical latency for a flash read or write. Second, AnyKey needs to *sort a set of KV entities based on the hash values of their keys*, when a data segment group is constructed during the compaction process. Note however that each of the groups to be merged contains already-sorted KV entities, resulting in a very short sorting time. Merging two data segment groups, each including sorted 8,192 KV entities, takes about 118us on the ARM Cortex-A53 core, which is negligible in our opinion, compared to the time-consuming compaction operation. We also want to emphasize that all the experimental data presented in Section 5 already include the computation overheads discussed above.

Table 2. Our tested KV workloads (sizes are in bytes).

Workload	Description	Key	Value
KVSSD[36]	The workload used in Samsung.	16	4,096
YCSB[17]	The default key and value sizes of YCSB.	20	1,000
W-PinK[31]	The workload used in PinK.	32	1,024
Xbox[19]	Xbox LIVE Primetime online game.	94	1,200
ETC[6]	General-purpose KV store of Facebook.	41	358
UDB[10]	Facebook storage layer for social graph.	27	127
Cache[58]	Twitter's cache cluster.	42	188
VAR[6]	Server-side browser info. of Facebook.	35	115
Crypto2[55]	Trezor's KV store for Bitcoin wallet.	37	110
Dedup[21]	DB of Microsoft's storage dedup. engine.	20	44
Cache15[59]	15% of the 153 cache clusters at Twitter.	38	38
ZippyDB[10]	Object metadata of Facebook store.	48	43
Crypto1[9]	BlockStream's store for Bitcoin explorer.	76	50
RTDATA[26]	IBM's real-time data analytics workloads.	24	10

4.5 An Enhancement to Compaction Process

4.5.1 Motivational Observations. We observed that, in general, while AnyKey exhibits significantly-improved IOPS under the low-v/k workloads, it experiences decreased IOPS for the high-v/k workloads, and also, even for the ones with relatively large values under the low-v/k workloads. According to our analysis, in those workloads, during a log-triggered compaction, a considerable volume of values are merged to the data segment groups. This in turn makes the target (destination) level reach its threshold, and invoke a tree-triggered compaction successively. We call such consecutive compaction invocations *compaction chain*. Figure 9a illustrates a situation where such a compaction chain occurs. ❶ When the size of the value log reaches its threshold, a log-triggered (L_{n-1}-to-L_n) compaction is invoked (note that L_{n-1} is selected as a target source level since the total size of its values is the largest), and hence, a large volume of its values are merged into the destination level L_n. ❷ During the compaction, the destination level L_n reaches its threshold, and consecutively, a tree-triggered (L_n-to-L_{n+1}) compaction is invoked.

4.5.2 Modified Log-Triggered Compaction. To avoid compaction chains, we add a simple yet effective enhancement to AnyKey, where the log-triggered compaction operation is slightly modified. Specifically, as described in Figure 9b, during a log-triggered (L_{n-1}-to-L_n) compaction, ❶ AnyKey monitors whether the size of the target destination level (L_n) reaches its threshold, (while merging the values in the value log to the data segment groups). ❷ Once the size of the target destination level reaches a certain point (i.e., $threshold \times \alpha$, $0 \leq \alpha \leq 1$; α can vary depending on the key/value sizes of the target workload), the remaining values (to be merged to the data segment groups) are written back into the value log. Doing so prevents any chance of a compaction chain from being generated.

However, this can decrease the benefit of a log-triggered compaction (i.e., reclaimed clean space in the value log can decrease), as values are written back to the value log after a certain point. To this end, we also change how to choose a target level. Instead of the level whose size of values in the value log is the largest, we select the one who has the most "invalid" values in the value log. In an LSM-tree, in general, there exist multiple different versions of values for a key, due to its out-of-place update mechanism, and clearly, the value log also contains a large number of invalid values. By selecting such a target level, we can further secure the available space in the value log while avoiding the problematic compaction chains.

5 Evaluation

5.1 Experimental Methodology

Testbed: We constructed a KV-SSD testbed based on a widely-used flash emulator (FEMU [45]).[8] One critical aspect of our testbed is to use the FIO plugin engine of the uNVMe driver [2] for generating requests of a target workload in the user space and making them directly destined to the emulated SSD *without* passing through the conventional S/W stack. We ported the publicly-available source code of PinK [31] into our testbed, and modified/re-engineered its data structures to implement AnyKey. The source code of our implementation is available at the following link: https://github.com/chanyoung/kvemu.

Compared systems: We compared the two versions of AnyKey against PinK. To distinguish the enhanced version of AnyKey (with the modified log-triggered compaction of Section 4.5.2) from its base version, we call it AnyKey+. To the best of our knowledge, PinK [30] stands out as the most advanced KV-SSD design to date.

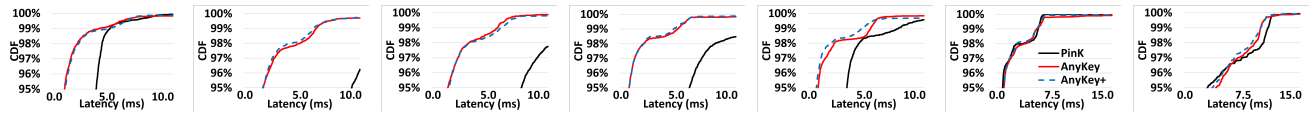
Device configuration: We assumed a 64GB SSD, which consists of eight channels, each including eight flash chips, as in PinK [31]. While, by default, the page size and the DRAM capacity are set, respectively, to 8KB and 64MB (note that, the DRAM capacity is usually set to 0.1% of the total SSD capacity [25, 39])[9], we also varied their sizes to investigate their impacts on our proposed design. Assuming a modern TLC flash, we set the read, write, and erase times to (56.5, 77.5, 106) us, (0.8, 2.2, 5.7) ms, and 3ms, respectively, as in [34]. Although we assume a small SSD due to the limitations of our emulation-based study, our target problem can still arise even with larger SSD sizes and correspondingly increased DRAM sizes, which makes AnyKey a viable solution. We also evaluated a 4TB SSD with a 4GB DRAM in Section 5.9.

Workloads: The 14 KV workloads we used in this work are listed in Table 2.[10] We set their write ratios to 20%. By default, we used the Zipfian distribution (θ of 0.99) for the key

[8] Considering the general performance of the processors in commodity SSDs, we set the clock frequency of the machine on which the emulation is performed to 1.2GHz, as in [31].

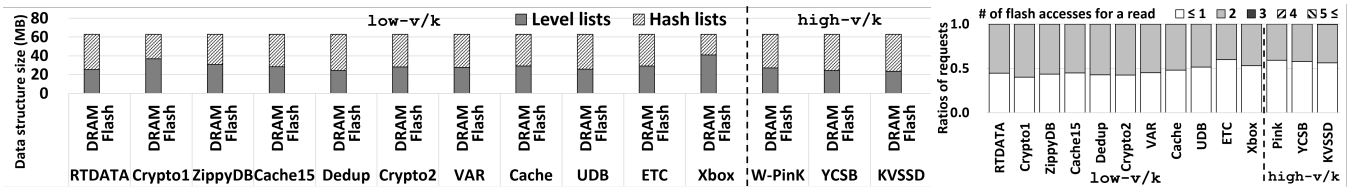
[9] We used the same configuration (e.g., page size and DRAM size) as in PinK to ensure an accurate and fair comparison.

[10] YCSB suites assume consistent key (20 bytes) and value sizes (1,000 bytes), and differentiate individual workloads by varying the ratios of insert/update/read/scan operations. To represent a conventional high-v/k



(a) RTDATA (b) Crypto1 (c) ZippyDB (d) Cache15 (e) Cache (f) W-PinK (g) KVSSD

Figure 10. The CDF of (95^{th} percentile) read latencies of the representative workloads under the three systems.



(a) The sizes and locations (DRAM or flash) of the data structures for the metadata. **(b)** The number of flash accesses for a read.

Figure 11. An analysis on the metadata size and its impact on AnyKey and AnyKey+.

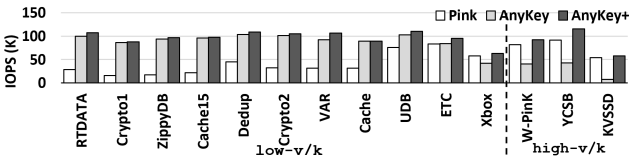


Figure 12. The IOPS achieved by the three systems.

distribution of each workload. However, we also varied it and evaluated its impact on our proposed design. The I/O queue depth is set to 64, which theoretically allows all 64 underlying flash chips to service I/O requests simultaneously. To the best of our knowledge, there are no publicly available KV applications to evaluate KV-SSD designs. Note that general KV applications cannot be used for this purpose, since they generate block I/O traces through the conventional full I/O stack. For this reason, KV-SSD research typically relies on KV generators (e.g., those based on the FIO engine) by adjusting various attributes such as key/value sizes, read/write ratios, and access distributions.

Execution procedure: After the warm-up stage, during which all KV pairs are inserted and GC/compaction operations are triggered frequently, we proceed to the execution stage, where we collect performance data. This stage concludes when the total request size reaches the full SSD capacity (i.e., 64GB) two times. For the workloads we tested, this process takes 4–13 hours.

5.2 Tail Latencies and IOPS

Figure 10 compares the 95^{th} percentile read tail latencies of the three systems under five low-v/k and two high-v/k workloads. For the low-v/k workloads, as shown in Figures 10(a)–(e), the tail latencies of PinK are quite long, while AnyKey and AnyKey+ reduce the tail latencies significantly.

workload, we maintain these key/value sizes while intentionally setting the update/read ratio to 20/80 for a fair comparison with other workloads.

Table 3. The analysis on compaction and GC operations.

Workload	Compared systems	The number of page reads or writes			
		Compaction		GC	
		Read	Write	Read	Write
Crypto1 (low-v/k)	PinK	85M	88M	340M	0
	AnyKey	49M	26M	0	0
	AnyKey+	49M	26M	0	0
Cache (low-v/k)	PinK	41M	45M	339M	0
	AnyKey	64M	20M	0	0
	AnyKey+	48M	15M	0	0
W-PinK (high-v/k)	PinK	3M	9M	27M	0
	AnyKey	25M	22M	0	0
	AnyKey+	20M	6M	26K	26K
KVSSD (high-v/k)	PinK	0.2M	10M	15M	0
	AnyKey	48M	50M	0	0
	AnyKey+	6M	9M	0	0

According to our analysis on AnyKey (Figure 11a), (i) the level lists are fully accommodated in the DRAM due to their extremely-reduced sizes, and (ii) even a lot of hash lists are generated and stored in the remaining DRAM space. Consequently, the number of flash accesses for a read can be limited to at most 2 in most cases (see Figure 11b). Recall that, if the target value is stored in the value log, AnyKey needs two flash accesses – one for the target KV entity in the data segment group, and the other for the value (which is pointed by the entity). Figures 10(f) and (g) indicate that AnyKey exhibits short latencies comparable to PinK under the high-v/k workloads, and thus, AnyKey is also available choice for them. Note that the structural design of AnyKey+ is the same as that of AnyKey; so, similar analysis results are observed.

Figure 12 shows the IOPS of the three systems tested. For the low-v/k workloads, AnyKey improves the IOPS by 3.15×, on average, compared to PinK. Such a substantial improvement originates from the two key factors: (i) the reduced flash access counts for read requests (as shown in Figure 11b), and (ii) the decreased compaction overhead (which will be further discussed in the following section). For the high-v/k workloads, AnyKey and PinK beat each other depending on

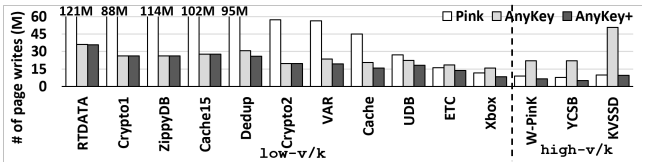


Figure 13. The total number of page writes experienced.

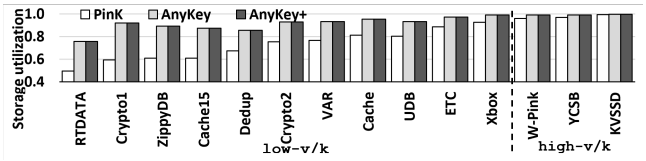


Figure 14. The storage utilizations of the three systems.

the workload, while AnyKey+ outperforms the both. This is because AnyKey+ avoids any possibility of experiencing compaction chains.

5.3 Analysis of Compaction and GC

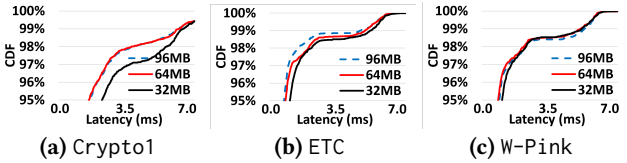
Table 3 lists the number of page reads/writes generated during the compaction/GC operations in the three systems.

- AnyKey and AnyKey+ experience no or few GC reads/writes. This is because the data segment groups of a level tend to be stored in the same blocks, and those groups in a block are relocated together during the compaction. As a result, no or few valid pages are found in the victim blocks.
- Under the low-v/k workloads, the read and write counts during the compaction in PinK increase significantly. This is mainly because, a large number of the meta segments have to be stored in the flash, which generates a lot of page reads and writes while being relocated during the compaction process. In contrast, AnyKey eliminates the meta segments, thereby avoiding the reads and writes originating from them.
- Under the high-v/k workloads, the compaction overheads in AnyKey can increase, if it encounters a compaction chain. In contrast, AnyKey+ reduces the overheads significantly by preventing compaction chains from forming.

5.4 Device Lifetime and Storage Utilization

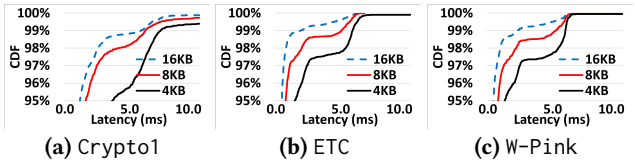
Figure 13 shows the total number of page writes experienced in each system, which actually indicates the compaction/GC writes. Recall that write requests are written into the flash during L_0-to-L_1 compaction. AnyKey+ reduces the number of writes by 50% on average, compared to PinK.

Figure 14 shows the storage utilization (the contribution of the total size of "unique" KV pairs to the entire storage capacity, when the storage is full). Under the low-v/k workloads, PinK needs to maintain a large number of the meta segments in the flash, which means that it stores two different copies for each key in the flash (one from the meta segment and the other from the data segment). In contrast, AnyKey and AnyKey+ always hold only one copy for each key in the flash by removing the meta segments.



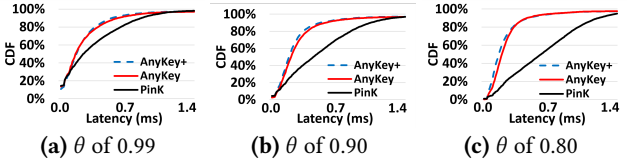
(a) Crypto1 **(b)** ETC **(c)** W-Pink

Figure 15. The read latencies under varying DRAM sizes.



(a) Crypto1 **(b)** ETC **(c)** W-Pink

Figure 16. The read latencies under varying flash page sizes.



(a) θ of 0.99 **(b)** θ of 0.90 **(c)** θ of 0.80

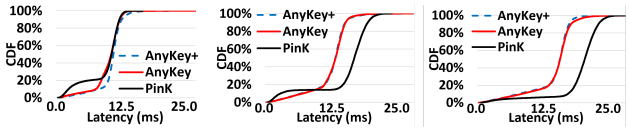
Figure 17. The lat. of ETC under varying key distributions.

5.5 Sensitivity Analysis

Different device settings can affect the benefits coming from AnyKey. First, we varied the DRAM capacity from 32MB to 96MB (0.05% to 0.15% of the entire SSD size), and observed the tail latencies experienced by AnyKey in three representative workloads (Figure 15). It can be seen that, under the low-v/k workloads where the metadata size is quite large, if the size of the DRAM decreases from 96MB to 32MB, the tail latencies can increase, since even a small amount of the metadata cannot be accommodated in the DRAM. Under the high-v/k workloads, the size of the metadata is usually small, and thus, a tiny (32MB) DRAM would be sufficient to hold it. Second, we varied the flash page size from 4KB to 16KB. As shown in Figure 16, the larger the page size, the shorter the tail latencies. Assuming that the total SSD size is fixed, as the page size increases (meaning the total number of flash pages decreases), the number of data segment groups in AnyKey decreases, which in turn means a reduced number of level list entries (metadata size).

5.6 Different Key Distributions

While we assumed a skewed pattern in the workloads by employing the Zipfian distribution with θ of 0.99, we also relaxed this assumption and evaluated our three tested systems under varying θ values (the lower the θ value, the more evenly distributed the keys). Figure 17 plots the latencies of ETC under different key distributions. As the key distribution becomes more even, the tail latencies of PinK get longer. This is because a number of requests in such workloads are destined into the lower levels of the LSM-tree, and their metadata are usually in the flash. In contrast, AnyKey and



(a) Length of 100 **(b)** Length of 150 **(c)** Length of 200

Figure 18. The latencies of UDB under varying scan lengths.

(a) IOPS. **(b)** Total number of page writes.

Figure 19. The Impact of varying the sizes of the value log.

AnyKey+ can find the metadata for such keys in the lower levels in the DRAM by reducing its size, and thus, offer more uniform performance across the different key distributions.

5.7 Range Queries

Since some KV workloads can issue "scan" requests, each retrieving tens to hundreds of consecutive keys, we prepared such a scan-centric workload based on UDB and used it to evaluate the three systems tested. Figure 18 compares the tail latencies of the three systems under varying lengths of the scan requests. It is to be observed that, more sub-requests each scan request includes, more benefits AnyKey and AnyKey+ can bring. This is because they manage a set of consecutive keys in the unit of a data segment group, which helps many sub-requests of a scan to be serviced from the same pages and groups. In contrast, the KV pairs in PinK are *not* sorted but scattered across the flash, which makes each sub-request potentially access a different flash page.

5.8 The Size of The Value Log

To evaluate the impact of the size of the value log, we performed a sensitivity analysis by varying it from 3.2GB to 9.6GB (5% to 15% of the 64GB SSD). Figure 19a plots the IOPS for ZippyDB, UDB, and ETC. When the value size is relatively small (ZippyDB), a small value log (e.g., 3.2GB) can achieve a high IOPS, which does not increase as the value log size increases. In contrast, as the value sizes increase (UDB and ETC), an increase in the value log size can lead to slightly higher IOPS. According to our analysis, if the value sizes of the workloads are small (e.g., low-v/k workloads), even a small value log would *seldom* become full, and thus *rarely* invokes a log-triggered compaction. Similar patterns can be also observed in the number of page writes (Figure 19b). We also considered a system where the value log is not introduced, called AnyKey-. According to our observations, (i) as the write ratio increases, AnyKey- decreases the IOPS significantly, whereas AnyKey+ still achieves high IOPS, and (ii) while AnyKey+ increases the tail latencies compared to AnyKey-, it is still comparable to PinK.

5.9 Design Scalability

While we assume a 64GB SSD, our proposal is applicable to any SSD capacity. Furthermore, the larger the SSD capacity (and thus, the more KV pairs the SSD includes), the more benefits one can expect from AnyKey, under the low-v/k workloads. For example, if Crypto1 is executed on a 4TB SSD, the size of the metadata in AnyKey is limited to 3.65GB, while that in PinK increases up to 25.2GB.

5.10 Multi-Workload Execution Scenarios

While we assume single-workload scenarios, it is also applicable to multi-workload scenarios. If one needs to co-locate two workloads, she can create two partitions [1, 29, 41], make each partition managed by an independent firmware (PinK or AnyKey), and execute each workload in its own partition. We conduct an experiment using a 2-workload scenario (W-Pink + ZippyDB) by creating two even partitions and managing the both with PinK or AnyKey. Compared to PinK, AnyKey can improve 99.9th percentile tail latencies of W-Pink and ZippyDB by 14% and 216%, respectively, indicating that using the partitioning mechanism for multi-workload scenarios can help each workload on a separate partition to get benefits coming from AnyKey.

6 Conclusions

We presented a novel KV-SSD design, called AnyKey, which is built upon our following observations: (i) the design principles of the existing KV-SSDs assume a limited range of workloads with extremely high value-to-key ratios, and (ii) there exist, in practice, other types of workloads with low value-to-key ratios. AnyKey, which targets such workload types, reduces additional flash accesses by minimizing the size of the metadata and helping it to be accommodated in the SSD-internal DRAM. Its enhanced form, which improves the management efficiency of the LSM-tree, outperforms the state-of-the-art for *both* ranges of workloads.

Acknowledgments

We would like to thank Asaf Cidon, our shepherd, and the anonymous reviewers for their valuable suggestions. This work was supported by NSF grants 2122155 and 1908793, as well as the Institute of Information & communications Technology Planning & Evaluation (IITP) grants funded by the Korean government (MSIT) (2022-0-00117, RS-2022-00155885, RS-2023-00221040). Wonil Choi is the corresponding author.

A Artifact Appendix

A.1 Abstract

The source code for our proposed and comparison systems, implemented in FEMU[11], is available at https://github.com/chanyoung/kvemu. Specifically, the source code for `PinK`, used as our comparison system, is located in the `hw/femu/pinkssd` directory, while the source code for `AnyKey`, our proposed system, is located in the `hw/femu/lksv3ssd` directory. Note that the two versions of our proposed system, `AnyKey` and `AnyKey+`, share the same source code, and users can specify which version to use by enabling the corresponding macro.

In a FEMU-based environment where a guest OS is executed, users can use their own images. However, we provide a modified Ubuntu image that includes uNVMe[12] and FIO[13], which work together as a key-value (KV) generator. To reproduce the results following the procedure in this artifact, users are requested to use our provided image, which can be downloaded via a web browser or with the `gdown` utility.

A.2 Artifact check-list (meta-information)

- **Program:** This artifact includes (i) FEMU code that implements both our proposed and comparison systems, and (ii) an Ubuntu image that can run on top of FEMU. Since our provided image includes a KV workload generator based on uNVMe and FIO, no additional software packages are required.
- **Data set:** The datasets and KV requests for our tested workloads are generated within this artifact using user commands in the provided Ubuntu image, which eliminates the need for external datasets or inputs.
- **Hardware:** To successfully emulate a 64GB SSD in terms of timing and functionality, we strongly recommend using a system with at least 10 cores and 256GB of DRAM. For our evaluation, we used a 1.2GHz 16-core processor with 256GB of DRAM.
- **Output:** As a result of the emulation, raw performance data are generated and saved in a log file. Using a script we provide, users can extract and retrieve only the relevant metric values from this log file.
- **How much disk space required (approximately)?:** Since FEMU uses DRAM space for SSD emulation, 50GB of disk space is sufficient for installing FEMU and downloading the OS image.
- **How much time is needed to prepare workflow (approximately)?:** To set up the environment for SSD emulation, users must download the source code and OS image, and then compile the source code. This process takes up to 10 minutes, though the exact time may vary depending on the user's environment.
- **How much time is needed to complete experiments (approximately)?:** Execution of the SSD emulation involves three stages. First is the data-filling stage, where data (key-value pairs) is loaded onto the emulated SSD; this process typically takes 5 to 150 minutes, depending on the target workload. Next is the warm-up stage, where the target workload runs for 60 to 150 minutes to stabilize SSD performance. Finally, in the data-collection stage, the workload runs until the cumulative size of issued key-value requests reaches 128GB (twice the size of the emulated 64GB SSD), during which performance results are collected. This final stage generally takes 180 to 480 minutes, depending on the workload. Overall, execution time can range from 245 to 780 minutes (up to 13 hours), depending on the target workload. Note also that the total execution time can vary significantly depending on the user's environment.

[11] https://github.com/MoatLab/FEMU
[12] https://github.com/OpenMPDK/unvme
[13] https://github.com/axboe/fio

A.3 Description

A.3.1 How to access.

- FEMU code: https://github.com/chanyoung/kvemu
- VM image: https://drive.google.com/file/d/1DJfaHnQpUn0pv0Tk2maNcmnwE8CFuKzN/view?usp=drive_link

A.3.2 Hardware dependencies.

- It is recommended to use a CPU with at least 10 cores and 256GB of DRAM.

A.3.3 Software dependencies.

- While uNVMe and FIO with the uNVMe plugin are required to generate each KV workload, they are already installed and ready to use in the provided Ubuntu image.
- Although there are some constraints for executing FEMU, such as OS compatibility, these can be avoided by using our provided Ubuntu image.

A.4 Installation

Users can download the provided Ubuntu image using the following steps:

```
# Assuming users are working in their home directory
cd; mkdir -p images; cd images

# Download it from our Google Drive using gdown
pip install gdown
gdown --id 1DJfaHnQpUn0pv0Tk2maNcmnwE8CFuKzN
gzip -d asplos.ubuntu.qcow2.gz
```

Next, users can download the source code and compile it using the following steps:

```
# Clone the artifact repository into the home directory
cd; git clone https://github.com/chanyoung/kvemu
cd kvemu

# Install the dependencies and build the source code
```

```
mkdir build-femu; cd build-femu
cp ../femu-scripts/femu-copy-scripts.sh .
./femu-copy-scripts.sh .
sudo ./pkgdep.sh
./femu-compile.sh
```

Note that when the source code is compiled, users can choose between the two versions of our proposed system: AnyKey and AnyKey+. Specifically, in the `include/hw/femu/kvssd/lksv/lksv3_ftl.h` file, users can select AnyKey by commenting out the macro for AnyKey+, and vice versa, as shown below:

```
#define OURS // for a AnyKey SSD
//#define OURS+ // for a AnyKey+ SSD
```

A.5 Experiment workflow

SSD emulation can be executed by running the following scripts: `run-pink.sh` for PinK, and `run-lksv3.sh` for AnyKey and AnyKey+ as shown below.

```
# Assuming users are in the build-femu directory
./run-pink.sh
./run-lksv3.sh
```

Once users have successfully executed the FEMU-based virtual system for SSD emulation, they need to open a terminal within the virtual system using the following command:

```
# The password of our provided image: liu
ssh liu@localhost -p 8080
```

Next, to prepare workloads, users need to set up uNVMe using the following commands.

```
cd uNVMe; sudo NRHUGE=256 ./script/setup.sh
```

Now, the system is ready to execute each workload on the emulated SSD. Workload execution involves running three different scripts in sequence, each corresponding to one of the execution stages mentioned earlier. For each target workload, dedicated scripts are available in `app/fio_plugin` and should be used accordingly. The following example demonstrates how to execute `etc` as the target workload.

```
cd app/fio_plugin
sudo ./fio-3.3 main/etc_load.fio
sudo ./fio-3.3 main/etc_ramp.fio
sudo ./fio-3.3 main/etc_run_20.fio

# Complete the execution by shutting down the VM
sudo shutdown -h now
```

Table 4. Three scripts for executing each of the tested workloads can be found in the `main` directory.

Workload name	Script for each stage		
	data-filling	warm-up	data-collection
KVSSD	kvssd_load.fio	kvssd_ramp.fio	kvssd_run_20.fio
YCSB	ycsb_load.fio	ycsb_ramp.fio	ycsb_run_20.fio
W-PinK	pink_load.fio	pink_ramp.fio	pink_run_20.fio
Xbox	xbox_load.fio	xbox_ramp.fio	xbox_run_20.fio
ETC	etc_load.fio	etc_ramp.fio	etc_run_20.fio
UDB	udb_load.fio	udb_ramp.fio	udb_run_20.fio
Cache	cache_load.fio	cache_ramp.fio	cache_run_20.fio
VAR	var_load.fio	var_ramp.fio	var_run_20.fio
Crypto2	crypto2_load.fio	crypto2_ramp.fio	crypto2_run_20.fio
Dedup	dedup_load.fio	dedup_ramp.fio	dedup_run_20.fio
Cache15	cache15_load.fio	cache15_ramp.fio	cache15_run_20.fio
ZippyDB	zippydb_load.fio	zippydb_ramp.fio	zippydb_run_20.fio
Crypto1	crypto1_load.fio	crypto1_ramp.fio	crypto1_run_20.fio
RTDATA	rtdata_load.fio	rtdata_ramp.fio	rtdata_run_20.fio

Table 4 lists each target workload along with its corresponding three scripts for execution.

A.6 Evaluation and expected results

After execution is complete, a `log` file is generated in the FEMU directory on the host machine. Users can extract various performance results from this log file by running the `extract.sh` script as shown below:

```
# Assume users are in the build-femu directory
./extract.sh log
```

References

[1] 2019. NVM Express Base Specification Revision 1.4. https://nvmexpress.org/wp-content/uploads/NVM-Express-1_4-2019.06.10-Ratified.pdf.

[2] 2020. uNVMe. https://github.com/OpenMPDK/uNVMe.

[3] 2022. Los Alamos National Laboratory and SK hynix to demonstrate first-of-a-kind Key-Value Store Computational Storage Device. https://discover.lanl.gov/news/0728-storage-device/.

[4] 2022. Why KV SSD will replace ZNS. https://www.snia.org/educational-library/why-kv-ssd-will-replace-zns-2022.

[5] ARM. 2020. Arm Storage Solution for SSD Controllers. (2020).

[6] Berk Atikoglu, Yuehai Xu, Eitan Frachtenberg, Song Jiang, and Mike Paleczny. 2012. Workload analysis of a large-scale key-value store. In *Proceedings of the 12th ACM SIGMETRICS/PERFORMANCE joint international conference on Measurement and Modeling of Computer Systems*. 53–64.

[7] Oana Balmau, Florin Dinu, Willy Zwaenepoel, Karan Gupta, Ravishankar Chandhiramoorthi, and Diego Didona. 2019. SILK: Preventing Latency Spikes in Log-Structured Merge Key-Value Stores. In *2019 USENIX Annual Technical Conference*. 753–766.

[8] Tim Bisson, Ke Chen, Changho Choi, Vijay Balakrishnan, and Yangsuk Kee. 2018. Crail-KV: A high-performance distributed key-value store leveraging native KV-SSDs over NVMe-oF. In *2018 IEEE 37th International Performance Computing and Communications Conference (IPCCC)*. IEEE, 1–8.

[9] BlockStream. [n. d.]. Electrs. https://github.com/Blockstream/electrs.

[10] Zhichao Cao, Siying Dong, Sagar Vemuri, and David H.C. Du. 2020. Characterizing, modeling, and benchmarking RocksDB key-value workloads at Facebook. In *18th USENIX Conference on File and Storage Technologies*.

[11] Hao Chen, Chaoyi Ruan, Cheng Li, Xiaosong Ma, and Yinlong Xu. 2021. SpanDB: A fast, Cost-Effective LSM-tree based KV store on hybrid storage. In *19th USENIX Conference on File and Storage Technologies*. 17–32.

[12] Youmin Chen, Youyou Lu, Fan Yang, Qing Wang, Yang Wang, and Jiwu Shu. 2020. Flatstore: An efficient log-structured key-value storage engine for persistent memory. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. 1077–1091.

[13] Chanwoo Chung, Jinhyung Koo, Junsu Im, Arvind, and Sungjin Lee. 2019. Lightstore: Software-defined network-attached key-value drives. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. 939–953.

[14] Yann Collet. 2016. xxHash: Extremely fast hash algorithm. https://github.com/Cyan4973/xxHash. (2016).

[15] Douglas Comer. 1979. Ubiquitous B-tree. *ACM Computing Surveys (CSUR)* 11, 2 (1979), 121–137.

[16] Alexander Conway, Abhishek Gupta, Vijay Chidambaram, Martin Farach-Colton, Richard Spillane, Amy Tai, and Rob Johnson. 2020. SplinterDB: Closing the bandwidth gap for NVMe Key-Value stores. In *2020 USENIX Annual Technical Conference*. 49–63.

[17] Brian F Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM symposium on Cloud computing*. 143–154.

[18] Yifan Dai, Yien Xu, Aishwarya Ganesan, Ramnatthan Alagappan, Brian Kroth, Andrea Arpaci-Dusseau, and Remzi Arpaci-Dusseau. 2020. From WiscKey to Bourbon: A Learned Index for Log-Structured Merge Trees. In *14th USENIX Symposium on Operating Systems Design and Implementation*. 155–171.

[19] Biplob Debnath, Sudipta Sengupta, and Jin Li. 2010. FlashStore: High throughput persistent key-value store. *Proceedings of the VLDB Endowment* 3, 1-2 (2010), 1414–1425.

[20] Biplob Debnath, Sudipta Sengupta, and Jin Li. 2011. SkimpyStash: RAM space skimpy key-value store on flash-based storage. In *Proceedings of the 2011 ACM SIGMOD International Conference on Management of data*. 25–36.

[21] Biplob K Debnath, Sudipta Sengupta, and Jin Li. 2010. ChunkStash: Speeding Up Inline Storage Deduplication Using Flash Memory.. In *USENIX annual technical conference*. 1–16.

[22] Siying Dong, Andrew Kryczka, Yanqin Jin, and Michael Stumm. 2021. Evolution of development priorities in key-value stores serving large-scale applications: The rocksdb experience. In *19th USENIX Conference on File and Storage Technologies*. 33–49.

[23] Zhuohui Duan, Jiabo Yao, Haikun Liu, Xiaofei Liao, Hai Jin, and Yu Zhang. 2023. Revisiting Log-Structured Merging for KV Stores in Hybrid Memory Systems. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 674–687.

[24] Carl Duffy, Jaehoon Shim, Sang-Hoon Kim, and Jin-Soo Kim. 2023. Dotori: A Key-Value SSD Based KV Store. *Proceedings of the VLDB Endowment* 16, 6 (2023), 1560–1572.

[25] Samsung Electronics. 2016. Samsung Introduces World's Largest Capacity (15.36TB) SSD for Enterprise Storage Systems. https://news.samsung.com/global/samsung-now-introducing-worlds-largest-capacity-15-36tb-ssd-for-enterprise-storage-systems.

[26] Rohan Gandhi, Aayush Gupta, Anna Povzner, Wendy Belluomini, and Tim Kaldewey. 2013. Mercury: Bringing efficiency to key-value stores. In *Proceedings of the 6th International Systems and Storage Conference*. 1–6.

[27] Google. 2011. Leveldb. https://github.com/google/leveldb.

[28] Huge Greenberg, John Bent, and Gary Grider. 2015. MDHIM: A Parallel Key/Value Framework for HPC. In *7th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 15)*. USENIX.

[29] Jian Huang, Anirudh Badam, Laura Caulfield, Suman Nath, Sudipta Sengupta, Bikash Sharma, and Moinuddin K Qureshi. 2017. FlashBlox: Achieving Both Performance Isolation and Uniform Lifetime for Virtualized SSDs. In *15th USENIX Conference on File and Storage Technologies*. 375–390.

[30] Junsu Im, Jinwook Bae, Chanwoo Chung, Arvind, and Sungjin Lee. 2021. Design of LSM-tree-based Key-value SSDs with Bounded Tails. *ACM Transactions on Storage* 17, 2 (2021), 1–27.

[31] Junsu Im, Jinwook Bae, Chanwoo Chung, Arvind Arvind, and Sungjin Lee. 2020. PinK: High-speed In-storage Key-value Store with Bounded Tails.. In *USENIX Annual Technical Conference*. 173–187.

[32] Yanqin Jin, Hung-Wei Tseng, Yannis Papakonstantinou, and Steven Swanson. 2017. KAML: A flexible, high-performance key-value SSD. In *2017 IEEE International Symposium on High Performance Computer Architecture*. IEEE, 373–384.

[33] Myoungsoo Jung and Mahmut Kandemir. 2012. An Evaluation of Different Page Allocation Strategies on High-Speed SSDs. In *4th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 12)*. USENIX.

[34] Myoungsoo Jung, Jie Zhang, Ahmed Abulila, Miryeong Kwon, Narges Shahidi, John Shalf, Nam Sung Kim, and Mahmut Kandemir. 2017. SimpleSSD: Modeling solid state drives for holistic system simulation. *IEEE Computer Architecture Letters* 17, 1 (2017), 37–41.

[35] Olzhas Kaiyrakhmet, Songyi Lee, Beomseok Nam, Sam H Noh, and Young-ri Choi. 2019. SLM-DB: Single-Level Key-Value store with persistent memory. In *17th USENIX Conference on File and Storage Technologies*. 191–205.

[36] Yangwook Kang, Rekha Pitchumani, Pratik Mishra, Yang-suk Kee, Francisco Londono, Sangyoon Oh, Jongyeol Lee, and Daniel DG Lee. 2019. Towards building a high-performance, scale-in key-value storage system. In *Proceedings of the 12th ACM International Conference on Systems and Storage*. 144–154.

[37] Hiwot Tadese Kassa, Jason Akers, Mrinmoy Ghosh, Zhichao Cao, Vaibhav Gogte, and Ronald Dreslinski. 2021. Improving performance

of flash based Key-Value stores using storage class memory as a volatile memory extension. In *2021 USENIX Annual Technical Conference*. 821–837.

[38] Yangseok Ki. 2017. Key Value SSD Explained - Concept, Device, System and Standard. https://www.snia.org/sites/default/files/SDC/2017/presentations/Object_ObjectDriveStorage/Ki_Yang_Seok_Key_Value_SSD_Explained_Concept_Device_System_and_Standard.pdf.

[39] Hyukjoong Kim, Dongkun Shin, Yun Ho Jeong, and Kyung Ho Kim. 2017. SHRD: Improving Spatial Locality in Flash Storage Accesses by Sequentializing in Host and Randomizing in Device. In *15th USENIX Conference on File and Storage Technologies (FAST 17)*. USENIX Association, Santa Clara, CA, 271–284. https://www.usenix.org/conference/fast17/technical-sessions/presentation/kim

[40] Tim Kraska, Alex Beutel, Ed H Chi, Jeffrey Dean, and Neoklis Polyzotis. 2018. The case for learned index structures. In *Proceedings of the 2018 international conference on management of data*. 489–504.

[41] Miryeong Kwon, Donghyun Gouk, Changrim Lee, Byounggeun Kim, Jooyoung Hwang, and Myoungsoo Jung. 2020. DC-Store: Eliminating noisy neighbor containers using deterministic I/O performance and resource isolation. In *18th USENIX Conference on File and Storage Technologies*. 183–191.

[42] Miryeong Kwon, Seungjun Lee, Hyunkyu Choi, Jooyoung Hwang, and Myoungsoo Jung. 2022. Vigil-KV: Hardware-Software Co-Design to Integrate Strong Latency Determinism into Log-Structured Merge Key-Value Stores. In *2022 USENIX Annual Technical Conference*. 755–772.

[43] Chang-Gyu Lee, Hyeongu Kang, Donggyu Park, Sungyong Park, Youngjae Kim, Jungki Noh, Woosuk Chung, and Kyoung Park. 2019. iLSM-SSD: An intelligent LSM-tree based key-value SSD for data analytics. In *2019 IEEE 27th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems*. IEEE, 384–395.

[44] Cheng Li, Hao Chen, Chaoyi Ruan, Xiaosong Ma, and Yinlong Xu. 2021. Leveraging nvme ssds for building a fast, cost-effective, lsm-tree-based kv store. *ACM Transactions on Storage* 17, 4 (2021), 1–29.

[45] Huaicheng Li, Mingzhe Hao, Michael Hao Tong, Swaminathan Sundararaman, Matias Bjørling, and Haryadi S Gunawi. 2018. The CASE of FEMU: Cheap, accurate, scalable and extensible flash emulator. In *16th USENIX Conference on File and Storage Technologies*. 83–90.

[46] Wenjie Li, Dejun Jiang, Jin Xiong, and Yungang Bao. 2020. HiLSM: an LSM-based key-value store for hybrid NVM-SSD storage systems. In *Proceedings of the 17th ACM International Conference on Computing Frontiers*. 208–216.

[47] Yuhong Liang, Tsun-Yu Yang, and Ming-Chang Yang. 2021. KVIMR: Key-Value Store Aware Data Management Middleware for Interlaced Magnetic Recording Based Hard Disk Drive. In *2021 USENIX Annual Technical Conference*. 657–671.

[48] Hyeontaek Lim, Bin Fan, David G Andersen, and Michael Kaminsky. 2011. SILT: A memory-efficient, high-performance key-value store. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*. 1–13.

[49] Lanyue Lu, Thanumalayan Sankaranarayana Pillai, Hariharan Gopalakrishnan, Andrea C Arpaci-Dusseau, and Remzi H Arpaci-Dusseau. 2017. Wisckey: Separating keys from values in ssd-conscious storage. *ACM Transactions on Storage* 13, 1 (2017), 1–28.

[50] Leonardo Marmol, Swaminathan Sundararaman, Nisha Talagala, and Raju Rangaswami. 2015. NVMKV: A Scalable, Lightweight, FTL-aware Key-Value Store.. In *USENIX Annual Technical Conference*. 207–219.

[51] Donghyun Min and Youngjae Kim. 2021. Isolating namespace and performance in key-value SSDs for multi-tenant environments. In *Proceedings of the 13th ACM Workshop on Hot Topics in Storage and File Systems*. 8–13.

[52] Patrick O'Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O'Neil. 1996. The log-structured merge-tree (LSM-tree). *Acta Informatica* 33 (1996), 351–385.

[53] SNIA. [n. d.]. KV Storage API spec. https://www.snia.org/keyvalue.

[54] Yongju Song, Wook-Hee Kim, Sumit Kumar Monga, Changwoo Min, and Young Ik Eom. 2023. Prism: Optimizing Key-Value Store for Modern Heterogeneous Storage Devices. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 588–602.

[55] Trezor. [n. d.]. BlockBook. https://github.com/trezor/blockbook.

[56] Jing Wang, Youyou Lu, Qing Wang, Minhui Xie, Keji Huang, and Jiwu Shu. 2022. Pacman: An Efficient Compaction Approach for Log-Structured Key-Value Store on Persistent Memory. In *2022 USENIX Annual Technical Conference*. 773–788.

[57] Sung-Ming Wu, Kai-Hsiang Lin, and Li-Pin Chang. 2018. KVSSD: Close integration of LSM trees and flash translation layer for write-efficient KV store. In *2018 Design, Automation & Test in Europe Conference & Exhibition*. IEEE, 563–568.

[58] Juncheng Yang. [n. d.]. Cache trace statistics. https://github.com/twitter/cache-trace/blob/master/stat/2020Mar.md.

[59] Juncheng Yang, Yao Yue, and KV Rashmi. 2020. A large scale analysis of hundreds of in-memory cache clusters at Twitter. In *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation*. 191–208.

[60] Ting Yao, Yiwen Zhang, Jiguang Wan, Qiu Cui, Liu Tang, Hong Jiang, Changsheng Xie, and Xubin He. 2020. MatrixKV: Reducing Write Stalls and Write Amplification in LSM-tree Based KV Stores with Matrix Container in NVM. In *2020 USENIX Annual Technical Conference*. 17–31.

[61] Chencheng Ye, Yuanchao Xu, Xipeng Shen, Xiaofei Liao, Hai Jin, and Yan Solihin. 2021. Hardware-based address-centric acceleration of key-value store. In *2021 IEEE International Symposium on High-Performance Computer Architecture*. IEEE, 736–748.

[62] Joohyeong Yoon, Won Seob Jeong, and Won Woo Ro. 2020. Check-in: In-storage checkpointing for key-value store system leveraging flash-based SSDs. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture*. IEEE, 693–706.

[63] Jiacheng Zhang, Youyou Lu, Jiwu Shu, and Xiongjun Qin. 2017. FlashKV: Accelerating KV performance with open-channel SSDs. *ACM Transactions on Embedded Computing Systems* 16, 5s (2017), 1–19.

[64] Yi Zheng, Joshua Fixelle, Nagadastagiri Challapalle, Pingyi Huo, Zhaoyan Shen, Zili Shao, Mircea Stan, and Vijaykrishnan Narayanan. 2022. ISKEVA: in-SSD key-value database engine for video analytics applications. In *Proceedings of the 23rd ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems*. 50–60.